

Huffman Coding: Greedy Compression, Proven Optimal

Programming and Data Structures — Week 10, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain why fixed-length character encoding wastes space when character frequencies are uneven, and define precisely what a prefix code is and why that property matters for unambiguous decoding; build a Huffman tree from scratch using Week 8's heap structure, and trace its construction completely on a concrete example; read the resulting variable-length binary codes directly off the constructed tree, and use them to encode and decode a real message; and explain, at a high level, why the greedy strategy of always merging the two least-frequent symbols is provably optimal.

Outline

Today covers a Morse-code analogy for variable-length encoding; fixed-length codes and the waste they introduce when frequencies are uneven, motivating prefix codes as the fix; building a Huffman tree from scratch, traced completely, directly reusing Week 8's heap; reading the resulting codes off the tree and using them to encode and decode a real message; the exchange argument establishing why greedy is correct here, genuinely different from Lecture 1's; computing the actual compression ratio achieved on a concrete example; the MTech-track material on Huffman coding's real-world deployment and its limits; an in-class quiz; and a summary.

The Morse-code analogy

Morse code assigns very short codes to common letters like E (a single dot) and T (a single dash), and considerably longer codes to rare letters like Q and Z. This is not an arbitrary design choice — it minimizes the average transmission time across real English text, given how frequently each letter actually occurs. Huffman coding is the rigorous, general algorithm for constructing exactly this kind of frequency-aware variable-length code, for any alphabet and any measured set of frequencies, not merely the specific case of English letters.

Fixed-length codes and their waste

Consider 5 symbols with frequencies A=45, B=13, C=12, D=16, E=14, out of 100 total characters. A fixed-length code must use $\lceil \log_2 5 \rceil = 3$ bits per symbol, regardless of frequency, giving $100 \times 3 = 300$ total bits for the whole message. But A occurs 45% of the time — spending the same 3 bits on A every single occurrence is wasteful if a shorter code for A specifically were possible, exactly the inefficiency Huffman coding is built to eliminate.

Prefix codes: the property that makes variable length work

A prefix code is one where no character's assigned code is a prefix of any other character's code. This property is what makes variable-length codes decodable at all without ambiguity: without it, encountering the bit sequence 01 when A is coded as 0 and B is coded as 01 would be genuinely ambiguous — is it A followed by the start of something else, or is it B? Huffman's tree-based construction guarantees a valid prefix code automatically, as a direct structural consequence of how codes are read off the tree, demonstrated in the slides that follow.

Building the Huffman tree: the greedy idea

The Huffman construction algorithm repeatedly takes the two currently least-frequent remaining nodes and merges them into a new internal node, whose frequency is the sum of the two merged nodes' frequencies. This newly created merged node re-enters the pool of candidates, and may itself be selected for a future merge if its combined frequency happens to still be among the smallest remaining. This process continues until exactly one node remains, which becomes the root of the completed Huffman tree.

Building the tree using Week 8's heap

```
import heapq

def build_huffman_tree(freqs):
    # freqs: dict of symbol -> frequency
    heap = [[freq, [symbol, ""]] for symbol, freq in freqs.items()]
    heapq.heapify(heap)                                # Week 8 Lecture 2's O(n) heapify
    while len(heap) > 1:
        lo = heapq.heappop(heap)                       # least-frequent node
        hi = heapq.heappop(heap)                       # second-least-frequent node
        for pair in lo[1:]: pair[1] = '0' + pair[1]    # prepend 0 to lo's codes
        for pair in hi[1:]: pair[1] = '1' + pair[1]    # prepend 1 to hi's codes
```

```
heapq.heappush(heap, [lo[0] + hi[0]] + lo[1:] + hi[1:])
return sorted(heap[0][1:], key=lambda p: (len(p[-1]), p))
```

This implementation directly reuses Week 8 Lecture 2's min-heap: the heap always provides $O(1)$ access to the currently least-frequent remaining node, which is exactly what the greedy merge step needs at every single iteration. Each merge consists of two `extract_min` calls and one `insert`, each costing $O(\log n)$ per Week 8's analysis, and the whole process runs $n - 1$ times (each merge reduces the pool by one). Total cost: $O(n \log n)$ — the same complexity class as Week 6's sorting algorithms, arising here for a structurally different reason.

Tracing Huffman tree construction, completely

Frequencies: A=45, B=13, C=12, D=16, E=14

Step 1: two smallest are C(12), B(13) → merge → new node BC(25)
Step 2: two smallest are E(14), D(16) → merge → new node DE(30)
Step 3: two smallest are BC(25), DE(30) → merge → new node BCDE(55)
Step 4: two smallest are A(45), BCDE(55) → merge → root(100)

Codes read off the tree:

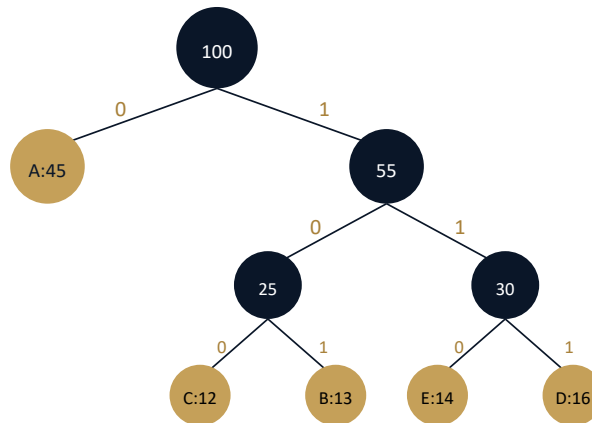
A: 0	(1 bit)
B: 101	(3 bits, via A BCDE → BC DE → C B)
C: 100	(3 bits)
D: 111	(3 bits, via BCDE → DE → D)
E: 110	(3 bits)

Tracing the full construction on the earlier frequency example: C and B, the two smallest at 12 and 13, merge first into a node of combined frequency 25. E and D, now the two smallest remaining, merge into a node of frequency 30. The two merged nodes, 25 and 30, are now themselves the two smallest, merging into a combined node of 55. Finally, A (45) and this combined node (55) merge into the root, frequency 100. Reading the codes off the resulting tree gives A a single bit (its high frequency earned the shortest code), and the other four symbols 3 bits each.

Reading codes off the tree, and computing the new total

Computing the total bits under this Huffman code: A contributes $45 \times 1 = 45$ bits, and each of B, C, D, E contributes 3 bits per occurrence, giving $13 \times 3 + 12 \times 3 + 16 \times 3 + 14 \times 3 = 39 + 36 + 48 + 42 = 165$, for a grand total of $45 + 165 = 210$ bits — compared to the fixed-length code's 300 bits. This is a genuine 30% reduction, and critically, Huffman's algorithm is provably optimal among all possible prefix codes for this exact frequency distribution — no other valid prefix code could achieve fewer total bits, a claim the following slides justify.

The picture: the completed Huffman tree



A, closest to the root, gets the shortest code (1 bit). B, C, D, E each get 3-bit codes.

Every leaf's code is read directly as the sequence of 0s and 1s labeling the edges from the root down to that leaf — A, sitting closest to the root, gets the shortest path and therefore the shortest code, while B, C, D, and E, all nested deeper, get progressively longer codes.

Encoding and decoding a message

```
codes = {"A": "0", "B": "101", "C": "100", "D": "111", "E": "110"}
message = "ABAC"
encoded = "".join(codes[ch] for ch in message)
print(encoded)    # "0" + "101" + "0" + "100" = "01010100"

def decode(bits, root_codes):
    reverse = {v: k for k, v in root_codes.items()}
    result, current = [], ""
    for bit in bits:
        current += bit
        if current in reverse:
            result.append(reverse[current])
            current = ""
    return "".join(result)
```

Encoding a message simply concatenates each character's Huffman code in order, with no separators of any kind needed between characters. Decoding reads the bit stream one bit at a time, checking

after each bit whether the bits accumulated so far form a complete, valid code for some symbol — the prefix-code property guarantees this check is always unambiguous, since no complete code can ever be a prefix of a different, longer code still being accumulated. This is the genuine payoff of the prefix property established earlier: decoding never needs to look ahead or guess at the boundary between two encoded characters.

Why greedy works here: a different exchange argument

The exchange argument justifying Huffman's greedy strategy differs in its specifics from Lecture 1's, though it uses the same underlying technique. The claim: in some optimal prefix code, the two least-frequent symbols are siblings positioned at the tree's maximum depth. If this were not already the case in some assumed-optimal tree, swapping these two least-frequent symbols with whichever two symbols *do* occupy the maximum-depth sibling positions cannot increase the total encoded cost — since assigning the longest code specifically to the least-frequent symbols, rather than to more frequent ones, only ever reduces or maintains the total weighted cost, never increases it. This establishes that merging the two least-frequent symbols first, exactly as this lecture's algorithm does, is always a safe, optimal first step.

MTech addition — real-world use, and Huffman's limits

Huffman coding is not merely a classroom exercise — the DEFLATE algorithm, used inside ZIP archives, gzip compression, and the PNG image format, uses Huffman coding as one explicit stage of its compression pipeline, directly building on this lecture's algorithm. Huffman's real limit is that it must assign each symbol a whole number of bits, since codes are read as paths through a binary tree; a symbol occurring 90% of the time would ideally deserve well under 1 bit on average, which Huffman's tree-based approach cannot express. Arithmetic coding and range coding, both beyond this course's scope, remove this whole-bit-per-symbol restriction entirely, achieving marginally better compression at the cost of considerably higher implementation complexity — a real trade-off, not a strictly superior replacement.

Tracing the decoder on the encoded message

```
encoded = "01010100"
reverse = {"0":"A", "101":"B", "100":"C", "111":"D", "110":"E"}

bit '0': current="0" → in reverse! → output "A", reset current=""
bit '1': current="1" → not in reverse
bit '0': current="10" → not in reverse
bit '1': current="101" → in reverse! → output "B", reset current=""
bit '0': current="0" → in reverse! → output "A", reset current=""
```

```
bit '1': current="1" → not in reverse
bit '0': current="10" → not in reverse
bit '0': current="100" → in reverse! → output "C", reset current=""
Decoded: "ABAC" - matches the original message exactly
```

Tracing the decoder bit by bit on "01010100": the first bit, 0, immediately matches A's code, decoding it and resetting. The next three bits accumulate to 101, matching B's code. The next single bit, 0, again matches A immediately. The final three bits accumulate to 100, matching C's code. The fully decoded result, "ABAC", exactly recovers the original message encoded earlier in this lecture, confirming the prefix-code property delivered on its promise of unambiguous decoding with no separators needed anywhere in the bit stream.

Exercise

As an exercise, build the Huffman tree by hand for the frequencies A=5, B=9, C=12, D=13, E=16, F=45 (a well-known six-symbol instance), and compute the total number of bits required for a message using these exact values as occurrence counts. Separately, implement the decode function from this lecture and confirm it correctly recovers the original message "ABAC" from its Huffman-encoded bit string, using the codes derived earlier in this lecture.

Connecting back: Huffman trees are binary trees (Week 7)

Every Huffman tree is a genuine binary tree in the sense of Week 7 — built from nodes with `left` and `right` children, constructed one merge at a time. Reading codes off the tree is structurally a form of traversal, following each root-to-leaf path and recording the edge label (0 for left, 1 for right) at every step, closely related to Week 7 Lecture 1's traversal orders. The tree's height equals the length of its longest code; unlike Week 7 Lecture 3's AVL trees, an unbalanced-looking Huffman tree is not a flaw to be corrected — it directly and correctly reflects genuinely uneven underlying frequencies, with rare symbols legitimately earning longer paths.

Quiz — is this a valid prefix code?

Given the codes A=0, B=10, C=110, D=111, determine before checking the answer whether this is a valid prefix code.

Quiz — answer

Checking every pair of codes for the prefix property: 0 is not a prefix of 10, 110, or 111 (they start with a different first bit). 10 is not a prefix of 110 or 111 (they differ at the first bit, since 1 versus 1 matches but the second bit 0 versus 1 differs). 110 and 111 differ only in their final bit, so neither is a prefix of the other. Every pair passes the check, confirming this is a valid prefix code — and indeed, this exact code shape (one 1-bit code, one 2-bit code, two 3-bit codes) is precisely what a genuine Huffman tree construction would produce for an appropriately skewed frequency distribution.

Comparing Huffman coding directly against Lecture 1

Placing both of this unit's greedy algorithms side by side highlights what genuinely differs and what does not: both rely on a proven exchange argument, both achieve $O(n \log n)$, but the specific greedy choice and the specific proof differ entirely, because they solve genuinely different problems. This is the concrete lesson underlying Lecture 1's warning against analogy: two correct greedy algorithms can share a cost class and a proof *technique* while requiring completely independent, problem-specific justifications.

Summary

Fixed-length character codes waste space whenever symbol frequencies are uneven, and prefix codes enable unambiguous variable-length encoding by guaranteeing no code is ever a prefix of another. Huffman's greedy construction — repeatedly merging the two currently least-frequent nodes — is built directly on Week 8's heap, costing $O(n \log n)$ total. A distinct exchange argument, genuinely different in its specifics from Lecture 1's, proves this greedy strategy optimal specifically for this problem. Real production systems — ZIP archives, gzip, and PNG images — genuinely use Huffman coding as an explicit compression stage, not merely as a textbook illustration. Next lecture introduces minimum spanning trees, this course's third greedy algorithm and its first genuine graph algorithm, previewing the full graph treatment beginning in Week 11.